Hướng dẫn sử dụng MS_Tool

Oct 17, 2026 · @SangNguyen

1. Giới thiệu

MS_Tool là phần mềm Windows để đo, phân tích bus CAN / CAN FD và chẩn đoán ECU theo UDS (ISO 14229) qua ISO-TP (ISO 15765-2). Tool chạy với Vector VN16xx (ví dụ VN1640) và PEAK PCAN-USB. Driver `vxlapi64.dll` và `PCANBasic.dll` đã năm sẵn trong thư mục `Lib` cạnh chương trình, không cần cài thêm vào Windows.

Bố cục màn hình

- **Thanh menu** ở trên cùng: File, Home, Hardware, Bus, Diagnostics, Tools, Test, Layout, Help.
- **Vùng làm việc** gồm các Desktop dạng tab ở đáy màn hình. Mỗi Desktop chứa các cửa sổ chức năng (Trace, Graphic, UDS Console…) xếp chồng hoặc lát cạnh nhau.
- **Thanh trạng thái** ở dưới cùng hiện trạng thái đo và thông tin license.

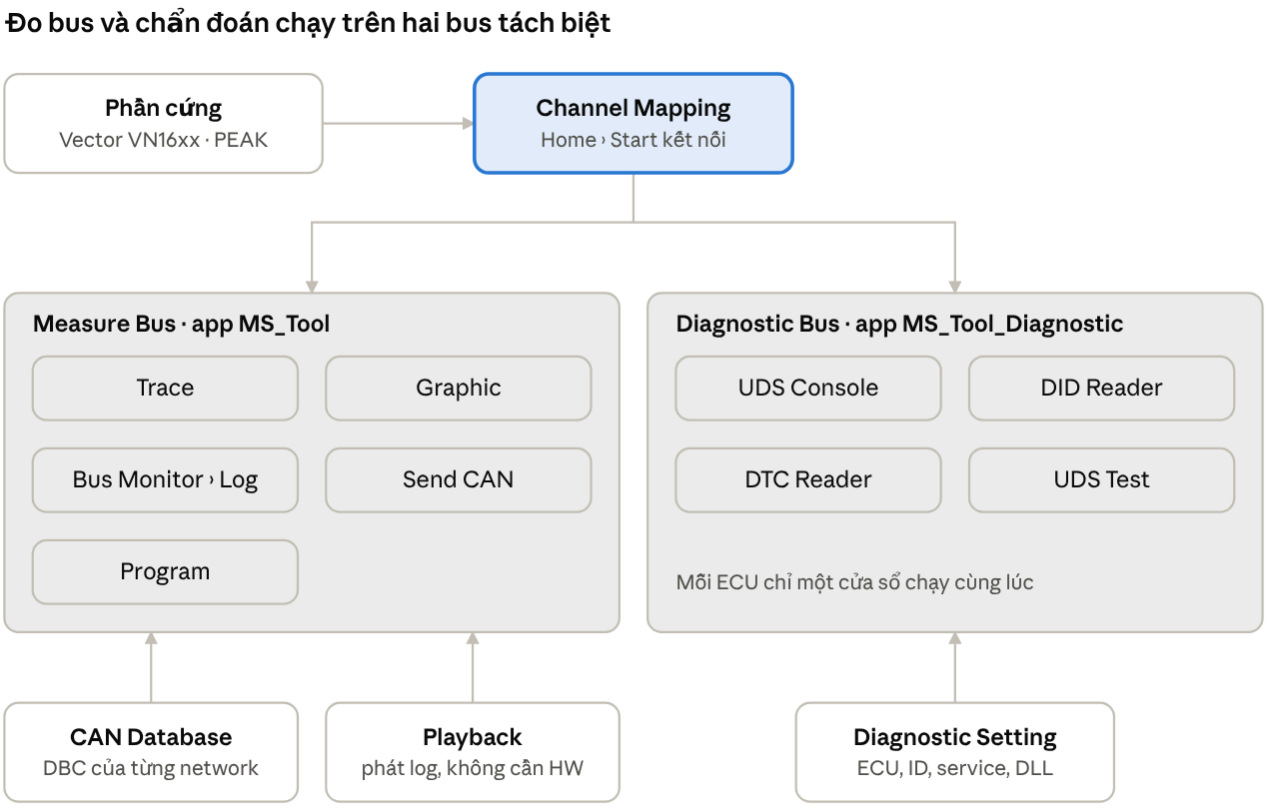
Menu và chức năng

Menu	Mục	Dùng để
File	New, Open, Save, Save As	Tạo, mở, lưu project
Home	Start, Stop, System Theme	Bắt đầu / dừng đo; chọn giao diện Light / Dark
Hardware	Hardware Configuration, Channel Mapping	Cài đặt thiết bị, tốc độ bus; gán network vào kênh phần cứng
Bus	Trace, Graphic, Bus Monitor (Log, Playback), Database › Add CAN Database, Transmit › Send CAN	Xem frame, vẽ signal, ghi / phát lại log, nạp DBC, gửi message
Diagnostics	Diagnostic Setting, UDS Console, DID Reader, DTC Reader	Cấu hình Program chẩn đoán, gửi lệnh UDS, đọc DID, đọc / xóa DTC
Tools	DBC Editor, CRC Calculator	Sửa file DBC; tính CRC
Test	UDS Test, Program	Chạy chuỗi test UDS; viết logic CAN bằng khối hoặc Python

Menu	Mục	Dùng để
Layout	Desktop, sắp xếp cửa sổ	Thêm / đổi tên / đóng Desktop; Cascade, Tile
Help	About, License…	Thông tin phiên bản; xem và nhập license key

Khái niệm cần biết

- **Project**: một thư mục lưu toàn bộ phiên làm việc (DBC đã nạp, mapping kênh, cửa sổ đang mở và cài đặt của chúng).
- **Network**: một mạng CAN có tên (ví dụ `PT`, `CHS`, `BD`) gắn với một file DBC trong CAN Database.
- **Measure Bus**: các bus dùng để đo và gửi frame. Trace, Graphic, Bus Monitor, Send CAN và Program dùng bus này.
- **Diagnostic Bus**: bus riêng cho chẩn đoán. UDS Console, DID Reader, DTC Reader và UDS Test dùng bus này.
- **Program (chẩn đoán)**: một bộ cấu hình theo dòng xe, gồm danh sách ECU, Req/Resp ID, service, file DTC và luật dịch ASCII. Tạo trong Diagnostic Setting.
- **Start / Stop** (menu Home): Start kết nối mọi network đang Active trong Channel Mapping; Stop ngắt tất cả. Cửa sổ mới mở luôn ở trạng thái dừng và chỉ chạy sau Start.



MS_Tool · hai bus và các cửa sổ dùng chúng

Channel Mapping gán mỗi network vào phần cứng và Home › Start mở cả hai bus. Playback thay phần cứng, đưa frame từ file log vào Measure Bus.

2. Khởi động và kích hoạt license

MS_Tool chỉ mở cửa sổ chính khi có license hợp lệ cho đúng máy đang dùng. Mỗi key gắn với Machine ID của máy, có ngày hết hạn và danh sách chức năng được mở.

Màn hình khởi động

Khi mở chương trình, màn hình splash hiện thanh tiến độ và tên từng bước nạp. Nếu tool mở chậm, thời gian từng bước được ghi trong `%APPDATA%\MS_Tool\logs\startup.log`; gửi file này kèm khi báo lỗi.

Kích hoạt lần đầu

1. Mở MS_Tool. Nếu chưa có key, hoặc key lỗi / hết hạn, màn hình **Kích hoạt** hiện ra kèm lý do.
2. Bấm **Copy** cạnh Machine ID và gửi mã này cho nhà cung cấp.
3. Nhận key (chuỗi ký tự) hoặc file `.lic`.

4. Dán key vào ô nhập, hoặc bấm **Mở file .lic** để chọn file. Key đúng thì tool vào cửa sổ chính.

Cài cho nhiều máy: đặt file `license.lic` cạnh file `.exe`, lần mở đầu tiên tool tự kích hoạt.

Xem và đổi license

- **Help › License…** hiện gói (level), ngày cấp, ngày hết hạn và danh sách chức năng ✔ / ✖. Có thể nhập key mới ngay tại đây.
- Khi license còn 14 ngày trở xuống, tool tự nhắc lúc khởi động.
- Machine ID không đổi khi cài lại Windows, vì lấy theo mainboard. Thay mainboard thì cần key mới.
- Chỉnh lùi đồng hồ máy để kéo dài hạn dùng sẽ bị phát hiện và license bị từ chối.

Chức năng bị khóa

Gói license gồm Basic, Standard, Pro, Full hoặc Custom (chọn từng chức năng). Mục menu không có trong license hiện **mờ kèm biểu tượng** 🔒 và không mở được; rê chuột lên để xem tên chức năng còn thiếu. Một số chức năng nằm bên trong cửa sổ cũng được khóa riêng: Graphic › Sequence Check, DID Reader › BOM Check, ghi DID (0x2E) và tự tính key Security Access.

3. Bắt đầu nhanh

Hai quy trình dưới đây đủ để đo bus và đọc chẩn đoán lần đầu. Chi tiết từng cửa sổ nằm ở các mục sau.

Đo và xem bus CAN

1. **File › New** bắt đầu project trống (đặt tên khi lưu lần đầu), hoặc **File › Open** mở project cũ.
2. **Bus › Database › Add CAN Database**: thêm network (ví dụ `PT`) và nạp file DBC cho từng network.
3. **Hardware › Hardware Configuration**: tick loại thiết bị (Vector, PEAK…), chọn CAN hoặc CAN FD, baudrate, rồi **Save**.
4. **Hardware › Channel Mapping**: bấm **Scan Devices**, ở bảng **Measure Bus** chọn thiết bị cho từng network và đặt Active = Yes.
5. Mở **Bus › Trace** (và **Bus › Graphic** nếu cần vẽ signal).
6. **Home › Start** để kết nối. Frame hiện trên Trace ngay.
7. Xong thì **Home › Stop**, rồi **File › Save** để lưu project.

Chẩn đoán UDS

1. **Diagnostics › Diagnostic Setting**: tạo Program (ví dụ `VF5`), nhập danh sách ECU từ Diag DBC, thêm service, chọn DLL Security Access nếu cần, rồi **Save**.
2. **Hardware › Channel Mapping**: ở bảng **Diagnostic Bus** chọn thiết bị cho network chẩn đoán và đặt Active = Yes.
3. **Hardware › Hardware Configuration**: kiểm tra P2, P2*, STmin và CAN type của phần Diagnostic.
4. **Home › Start**.
5. Mở **UDS Console** để gửi lệnh tay, **DID Reader** để đọc thông tin ECU, hoặc **DTC Reader** để đọc / xóa lỗi.

Không dùng UDS Console, DID Reader, DTC Reader hay UDS Test cùng lúc trên **cùng một ECU**: mỗi cửa sổ có engine riêng và chúng sẽ tranh nhau phản hồi.

4. Project, Desktop và giao diện

Project lưu lại cả phiên làm việc, nên mở lại project là có ngay network, DBC, mapping kênh và các cửa sổ như lúc lưu.

Project (menu File)

Lệnh	Phím tắt	Tác dụng
New	Ctrl+N	Bắt đầu project trống (Untitled). Có thay đổi chưa lưu thì tool hỏi Save / Discard / Cancel
Open	Ctrl+O	Danh sách project trong thư mục Projects và các project mở gần đây; **Browse…** để chọn project ở nơi khác
Save	Ctrl+S	Lưu project đang mở. Project chưa có tên thì hỏi tên như Save As
Save As	Ctrl+Shift+S	Lưu thành project mới; trùng tên thì tool hỏi trước khi ghi đè

Thanh tiêu đề hiện tên project, kèm dấu `*` khi có thay đổi chưa lưu. Khi khởi động, tool tự mở lại project dùng gần nhất.

Project lưu: các network và đường dẫn DBC, mapping kênh, các cửa sổ đang mở cùng cài đặt của từng cửa sổ (ví dụ hàng đợi Send CAN, bố cục DBC Editor). Dữ liệu chẩn đoán (Program) và thư viện Sequence Check được lưu riêng, dùng chung cho mọi project (xem Phụ lục).

Desktop (menu Layout)

- Mỗi tab ở đáy màn hình là một Desktop. Dùng nhiều Desktop để tách công việc, ví dụ một Desktop cho đo bus, một Desktop cho chẩn đoán.
- Layout cho phép thêm, đổi tên, đóng Desktop và sắp xếp cửa sổ (Cascade, Tile, thu nhỏ tất cả).
- Mỗi loại cửa sổ chỉ mở một lần trên một Desktop. Chọn lại menu sẽ đưa cửa sổ đang có lên trước.

Giao diện Light / Dark

Home › System Theme › Light Mode / Dark Mode đổi giao diện ngay cho toàn bộ cửa sổ đang mở. Lựa chọn được nhớ cho lần mở sau và không lưu trong project.

5. Phần cứng và đo (Hardware, Home)

Phần cứng được cài qua hai hộp thoại: Hardware Configuration chọn thiết bị và thông số bus, Channel Mapping gán từng network vào một kênh vật lý. Không có nút Connect riêng: **Home › Start** kết nối theo mapping đã lưu.

Hardware Configuration

Nhóm	Thông số	Ghi chú
Thiết bị	Vector, PEAK, SocketCAN, Virtual…	Chỉ loại được tick mới được Scan. Mặc định bật Vector và PEAK
Bus Setting	Type (CAN / CAN FD), Baudrate, FD Data Rate	FD Data Rate chỉ mở khi Type = CAN FD
Bus Setting	Vector App Name	Mặc định `MS_Tool`, dùng cho Trace / Send CAN
Diagnostic	P2, P2* (ms)	Thời gian chờ phản hồi thường và khi ECU báo NRC 0x78
Diagnostic	STmin	Nhận 0 – 127 ms (số nguyên) hoặc 0.1 – 0.9 ms (bước 0.1) theo ISO 15765-2
Diagnostic	CAN type	Bị khóa ở CAN nếu Bus Setting là CAN
Diagnostic	App Name	Mặc định `MS_Tool_Diagnostic`, tên app Vector riêng cho chẩn đoán

Bấm **Save** để lưu; kết quả ghi vào System Messages. Đổi thông số khi đang chạy chỉ có hiệu lực ở lần Start sau.

Channel Mapping

Hộp thoại có hai bảng: **Measure Bus** (đo, gửi frame) và **Diagnostic Bus** (chẩn đoán). Mỗi hàng là một network trong CAN Database, gồm cột Network, Bus CH, Active và Hardware.

- **Scan Devices** dò lại phần cứng theo các loại đã tick. Lần mở đầu tiên tool tự scan.
- **Refresh Network** đọc lại danh sách network từ CAN Database.
- Chọn thiết bị trong cột Hardware và đặt Active = Yes cho network cần dùng. Thay đổi được lưu ngay, không cần bấm OK.
- Hai thiết bị giống hệt nhau được đánh số để phân biệt, ví dụ `[1] VN1640 1` và `[2] VN1640 1`.
- Khi đang chạy, hộp thoại chỉ đọc. Bấm **Home › Stop** trước khi sửa.

Quy tắc dùng chung kênh: một kênh **Vector** dùng được ở cả hai bảng, vì tool mở hai app Vector riêng (`MS_Tool` và `MS_Tool_Diagnostic`). Thiết bị khác (PEAK, Virtual…) chỉ được giữ **một** vai trò: đã dùng ở bảng này thì hiện xám ở bảng kia, rê chuột lên mục xám để xem lý do.

Start / Stop

- **Home › Start** mở các network Measure Bus trước, rồi Diagnostic Bus. Network nào không tìm thấy phần cứng (chưa cắm) bị bỏ qua và có cảnh báo; các network khác vẫn chạy.
- Nếu dữ liệu cũ vi phạm quy tắc dùng chung kênh, Start bỏ qua network chẩn đoán đó và ghi lý do vào System Messages.
- **Home › Stop** ngắt Diagnostic Bus trước, rồi Measure Bus. Các cửa sổ đang chạy tự dừng.
- Start bị khóa trong lúc Bus Monitor đang Playback một file log.

System Messages

Cửa sổ System Messages là bảng Time / Message ghi mọi sự kiện của hệ thống: kết quả Save, Start / Stop, network bị bỏ qua, lỗi bus. Một lỗi bus lặp lại chỉ được ghi tối đa một lần mỗi 5 giây.

6. Bus

Các cửa sổ trong menu Bus làm việc trên Measure Bus và dùng DBC của CAN Database để giải mã signal. Trừ CAN Database, mọi cửa sổ mở ra ở trạng thái dừng và chỉ nhận frame sau **Home › Start** (hoặc khi Playback đang phát).

6.1 CAN Database (Bus › Database › Add CAN Database)

- Thêm network, đặt tên (ví dụ `PT`, `CHS`, `BD`) và nạp file DBC cho từng network. File lớn được nạp ở nền, giao diện không bị đơ.
- Cây bên trái hiện Network › Node › message TX / RX; bảng bên phải hiện đủ thuộc tính signal (start bit, length, byte order, factor, offset, đơn vị, value table…).
- Ô tìm kiếm lọc theo tên hoặc ID; không có kết quả thì ô viền đỏ.
- Đổi tên network trực tiếp trong danh sách. Không đổi tên hay xóa được network khi đang chạy hoặc đang Playback.
- Trace, Graphic, Send CAN, Channel Mapping và Program đều lấy network từ đây, nên đổi DBC ở đây là các cửa sổ khác cập nhật theo.

6.2 Trace (Bus › Trace)

Trace hiện frame của mọi network đang chạy, mỗi ID một dòng (cập nhật tại chỗ). Mở rộng một dòng để xem các signal đã giải mã theo DBC.

Nút	Tác dụng
Pause / Resume	Dừng cập nhật màn hình để đọc; frame vẫn được nhận và hiện lại khi Resume
Save	Xuất bảng ra file CSV
Clear	Xóa toàn bộ dòng
Auto-scroll	Tự cuộn theo dòng mới
Expand all / Collapse all	Mở / đóng signal của mọi dòng
Snapshot	Chụp nội dung hiện tại ra file
Settings	Chọn bộ cột theo giao thức (CAN, CAN FD, J1939, CANopen) và các tùy chọn hiển thị
Filter String	Lọc theo ID hoặc tên message

Cột FPS cho biết số frame mỗi giây của từng ID. Dòng ngừng cập nhật một lúc sẽ chuyển màu xám, giúp nhận ra ECU đã thôi gửi. Cài đặt cột và bộ lọc được nhớ cho lần sau.

6.3 Graphic (Bus › Graphic)

Graphic vẽ signal theo thời gian. Bên trái là danh sách signal, bên phải là vùng vẽ.

Thêm và quản lý signal

- **Add Signal** mở hộp chọn signal từ CAN Database. **Remove** / **Remove All** để bớt signal.
- Signal được xếp theo nhóm; kéo thả để đổi nhóm, ẩn / hiện cả nhóm.
- Đổi màu, ẩn / hiện từng signal, sửa dải trục Y ngay trong bảng.
- Signal có value table hiện giá trị dạng chữ và được vẽ dạng bậc thang.
- **Options** lưu / nạp bộ signal đang vẽ để dùng lại; signal không còn trong DBC vẫn giữ dòng và tô xám.

Vẽ và đo

Nút	Tác dụng
Run / Pause	Run bắt đầu ghi và cuộn theo thời gian thực. Pause đứng hình để soi, dữ liệu vẫn được ghi; Run lại thì nhảy về hiện tại
Overlay / Separate	Vẽ chung một vùng (mỗi signal một trục Y theo màu) hoặc mỗi signal một dải riêng
Select Mode	Chỉ vẽ signal đang chọn
Fit X / Fit Y, Zoom In / Out X	Căn khung và phóng trục thời gian
Undo / Redo view	Quay lại khung nhìn trước / sau
Cursor 1 / Cursor 2	Một cursor đọc giá trị tại thời điểm; hai cursor thêm cột Value2 và ΔY
Show Points	Hiện điểm mẫu trên đường vẽ
Clear	Xóa dữ liệu đã ghi
Tools	Xuất dữ liệu ra CSV hoặc ảnh PNG

Chọn một signal trong bảng thì đường của nó đậm hơn và điểm to hơn. Có thể thêm **comment theo thời gian thực**, hiện thành vạch dọc kèm nhãn trên vùng vẽ. Sau Home › Stop, dữ liệu được giữ để phân tích; lần Start sau sẽ xóa và lấy mốc thời gian mới.

Tab Configurations: Sequence Check

Tab này chỉ dùng để định nghĩa logic kiểm tra trình tự hoạt động của xe (HV ON / OFF, sạc, LV top-up, chạy…). Mỗi sequence là một file JSON trong thư viện chung `Sequence Check`, gồm:

- **Trigger**: nhóm điều kiện; mỗi lần nhóm này chuyển từ sai sang đúng là bắt đầu một lần kiểm tra. Có thể đặt thời gian giữ (hold) và cách xử lý khi trigger lặp lại (Ignore, Restart, Parallel).
- **Steps**: các bước theo thứ tự. Mỗi bước là nhóm điều kiện AND / OR phải đạt trong khoảng [min, max] ms tính từ trigger, bước trước, hoặc một bước chỉ định; hold ms chống nhiễu; bước optional không đạt cũng không tính lỗi.
- **Invariants**: điều kiện phải đúng suốt quá trình (ví dụ không có lỗi cách điện).
- **Abort**: điều kiện hủy lần kiểm tra (ví dụ tắt khóa giữa chừng); kết quả là ABORTED, không tính FAIL.
- **Timeout**: thời gian tối đa của một lần kiểm tra (0 = tới trigger kế tiếp hoặc hết log).

Điều kiện = signal + phép so sánh + giá trị. Phép so sánh: `==`, `!=`, `>`, `>=`, `<`, `<=`, `between`, `outside`, `one of`. Giá trị là số hoặc tên trong value table của DBC (ví dụ `CLOSED`); có thể đặt sai số cho phép. Nút **Validate** kiểm signal có trong DBC không và các mốc thời gian có hợp lý không. Việc chạy kiểm tra trên file log được thực hiện ở Bus Monitor (mục 6.4).

6.4 Bus Monitor (Bus › Bus Monitor, Log, Playback)

Bus Monitor có hai tab: **Log** để ghi bus ra file, **Playback** để mở và phát lại file log mà không cần phần cứng.

Tab Log

- Ghi frame của các network đang chạy ra file **BLF** (mặc định), **ASC** hoặc **TXT** (định dạng của MS_Tool).
- Mỗi file log có kèm file `.meta.json` lưu tên network của từng kênh, giúp Playback tự gợi ý mapping sau này.
- Bus chẩn đoán cũng được ghi để phân tích sau.
- Khi một kênh Vector dùng chung cho đo và chẩn đoán, frame trùng được lọc, không ghi hai lần.
- File BLF / ASC mở được trong CANoe và TSMaster.

Tab Playback

1. **Add** hoặc kéo thả file log vào bảng. Hỗ trợ BLF, ASC, TXT của MS_Tool, TRC (PEAK), CSV, candump `.log`, MF4 và DB. Nhiều file được phát nối tiếp theo thứ tự trong bảng (Move up / down để đổi).
2. Mỗi file được quét ở nền để lấy thời lượng, số frame và các ID theo từng kênh.
3. **Channel mapping**: gán kênh trong log vào network của project. Tool tự gợi ý dựa trên `.meta.json`, tỉ lệ ID khớp DBC và số kênh. Chọn `(ignore)` để bỏ một kênh.
4. **Range**: kéo hai đầu trên thanh thời gian để chỉ phát đoạn A → B.

5. **Speed**: Fast hoặc x0.5 / x1 / x2 / x5 / x10 thời gian thực; đổi được ngay khi đang phát.
6. Bấm phát. Trace và Graphic đang mở tự chạy và hiện dữ liệu như đo thật; mở thêm Trace / Graphic lúc đang phát cũng tự chạy theo.

Trong lúc Playback: Home › Start và CAN Database bị khóa; UDS Console, Send CAN, Program và tab Log **không** nhận frame phát lại, nên không có gì được gửi ra bus. Không phát được khi đang đo thật. Phát xong, bấm Stop hoặc đóng Bus Monitor là mọi thứ về trạng thái dừng.

Post process (trên file đã nạp): cắt đoạn đã chọn ra file BLF mới, thống kê chu kỳ từng ID ra CSV, và chạy Sequence Check đã định nghĩa ở Graphic.

6.5 Send CAN (Bus › Transmit › Send CAN)

Send CAN gửi message theo DBC hoặc message tự tạo, theo chu kỳ hoặc gửi tay, kèm CRC và Alive Counter.

1. Chọn network bằng các nút ở trên cùng. Message luôn được gửi trên đúng bus của network đó.
2. Tìm message trong danh sách bên trái, bấm **Add Selected** để đưa vào hàng đợi TX. **New Msg** tạo message tay (ID mở rộng ghi thêm `x`, ví dụ `18DAF110x`).
3. Trong **Signal editor**, sửa giá trị Raw hoặc Physical và chọn Generator cho từng signal. Ô xem trước hiện dữ liệu byte sẽ gửi.
4. Trong **TX Queue**, đặt Cycle (ms), Mode (Cyclic / Manual), BRS (CAN FD) và tick Active.
5. **E2E Setting** chọn kiểu CRC và Alive Counter (có tùy chọn tự định nghĩa), áp dụng cho các signal Checksum / Alive Counter.
6. Bấm **SEND**. Message Cyclic gửi đều theo chu kỳ; message Manual gửi một lần khi bấm SEND hoặc **Send Once**. **STOP** dừng tất cả.

Generator	Giá trị signal
None	Giữ giá trị đã nhập
CheckSum	Tính CRC theo E2E Setting mỗi lần gửi
Alive Counter	Tăng dần mỗi lần gửi rồi quay vòng
Ramp	Tăng dần theo bước
Random	Ngẫu nhiên trong dải của signal

SEND chỉ bật khi đã Start và network đã mở. Home › Stop dừng gửi; Start lại thì phải bấm SEND lại. Frame trễ hạn bị bỏ qua và đếm "late", không gửi dồn. **Export / Import** lưu hàng đợi cùng E2E Setting ra file `.json` để chia sẻ; khi Import, tool hỏi cách gán network.

7. Chẩn đoán (Diagnostics)

Mọi cửa sổ chẩn đoán lấy ECU, ID và service từ một **Program** tạo trong Diagnostic Setting, và chạy trên Diagnostic Bus đã map trong Channel Mapping. Program không chứa thông tin bus: chọn bus và CAN type ở phần Hardware.

7.1 Diagnostic Setting

Thanh trên cùng chọn Program, tạo Program mới (gõ tên rồi bấm **New**) và **Save**. Save ghi toàn bộ Program; nút **Generate / Export** trong từng tab chỉ ghi phần của tab đó. Dòng trạng thái báo phần nào chưa lưu (ví dụ `● Unsaved: ECU IDs, Services`); đổi Program hoặc đóng cửa sổ khi chưa lưu thì tool hỏi Save / Discard / Cancel.

Tab ECU IDs

- **AddDBC** nhập ECU từ Diag DBC: ECU `BCM` lấy từ message `BCM_DiagReq` / `BCM_DiagResp`; ID 29-bit nhận ra qua cờ extended. Message `Functional_DiagReq` (hoặc ID 0x7DF) được hiểu là ID chức năng, không tạo ECU.
- Nhập DBC lần nữa chỉ cập nhật ID của ECU đã có, giữ nguyên DLL và service; không tạo dòng trùng.
- **AddRow / RemoveRow** thêm hoặc bớt ECU. ECU mới có sẵn các function cơ bản.
- Sửa trực tiếp tên, Req ID, Resp ID, đường dẫn DLL. ID tự chuẩn hóa (`7a0` → `0x7A0`). Ô sai tô đỏ, tên trùng tô cam, DLL không tồn tại tô đỏ. Chuột phải ô DLL để **Browse**; đường dẫn tương đối tính từ thư mục Program.

Tab Services

- Mỗi ECU có sẵn function cơ bản: PN, HW, SW, Boot.
- **Add…** mở hộp thoại để bật / tắt function, thêm function từ template (CAL, HMI, TBOX…), thêm function tự đặt tên, hoặc xóa.
- Request gõ `F187`, `22F187` hay `22 F1 87` đều được chuẩn hóa thành `22F187`. Function có revision thì nhập thêm request revision; DID Reader sẽ hiện `<main>-<revision>`.
- **Add function › Import template** nạp danh sách function từ file `Template DID.xlsx`.

Tab DTC

- Double-click một dòng ECU để chọn file ODX dạng Excel (`.xlsx` / `.xlsm`).
- **Generate** đọc sheet `DTC list` (DTC number, tên lỗi, repair action) và ghi `<ECU>_DTC.json`. Việc đọc chạy ở nền.
- Cột Status tô màu theo trạng thái (Pending, Ready, Done, Error). Mở lại Program sẽ thấy `On disk (n records)` cùng đường dẫn file Excel đã dùng.

Tab ASCII Trans Rule

Bảng này quyết định byte nào của phản hồi được dịch sang chữ. Mỗi hàng là một Sub_Func (DID), mỗi cột là một byte, tick ô để dịch byte đó.

- **Byte#0** là byte giá trị đầu tiên sau DID. Ví dụ `22 F1 88` → `62 F1 88 01 02 03` thì Byte#0 = `01`.
- **Add Sub_Func**, **Remove**, **Add Byte** để mở rộng bảng theo hàng và cột. Double-click tiêu đề cột để tick / bỏ cả cột.
- Combo **ECU Apply**: `All` áp dụng cho mọi ECU; chọn một ECU để đặt luật riêng. Luật của ECU ghi đè luật `All` cho cùng Sub_Func.
- **Export** ghi file luật ASCII của Program.

Thứ tự làm đề xuất: tạo Program → ECU IDs (AddDBC, sửa DLL) → Services → DTC (Generate) → ASCII → Save.

7.2 UDS Console (Diagnostics › UDS Console)

UDS Console gửi lệnh UDS bằng tay tới một ECU và hiện phản hồi đã giải mã.

Chọn đích

- Chọn **Program**, rồi **ECU**. Req / Resp ID (chỉ đọc) và DLL Security Access tự điền theo Program; ID 29-bit được gửi đúng định dạng.
- Chọn `(Manual)` để tự gõ Req / Resp ID.
- Console dùng network **đầu tiên đang chạy** trên Diagnostic Bus; có nhiều network thì tool cảnh báo. P2, P2*, STmin và CAN type lấy từ Hardware Configuration.
- Chưa Start hoặc chưa có Diagnostic Bus thì nút Send xám và Console ghi rõ lý do.

Gửi lệnh

- Gõ hex theo cách nào cũng được: `22 F1 90`, `22F190` hoặc `0x22,0xF1,0x90`, rồi Enter hoặc **Send**. Hex sai được báo ngay (ví dụ "odd number of hex digits").
- Phím Lên / Xuống duyệt lại lịch sử lệnh.
- Gõ `?` hoặc `info` để in toàn bộ cấu hình hiện tại: Program, ECU và ID, bus, P2 / P2* / STmin, trạng thái Security, Tester Present, State. Khối này cũng tự in khi đổi Program / ECU và khi Start / Stop.

Đọc kết quả

- Khung **Console** hiện request / response UDS, Security Access, cảnh báo và lỗi theo màu. NRC hiện đủ chữ, ví dụ `Request Out Of Range (0x31)`.
- Phản hồi đọc DID được in kèm bản dịch ASCII theo luật của Program và thời gian phản hồi, ví dụ `62 F1 88 41 42 43 44 45 | ABCDE (23 ms)`.
- Khung **ISO-TP log** bên phải hiện các khung CAN thô (📤 gửi, 📥 nhận), gồm First Frame, Consecutive Frame, Flow Control.

Tester Present: tick để gửi `3E` mỗi 2000 ms tới Req ID vật lý của ECU đang chọn. Đổi sub-function `0x00` / `0x80` có hiệu lực ngay. Tester Present tạm dừng trong lúc có request đang chạy.

Security Access: khi ECU có DLL, gửi `27 01` (seed) là tool tự tính key bằng DLL và tự gửi `27 02`. Nếu DLL không tìm thấy hoặc bộ tính key chưa sẵn sàng, Console nêu lý do cụ thể.

7.3 DID Reader (Diagnostics › DID Reader)

DID Reader đọc lần lượt các DID của nhiều ECU (Part Number, HW, SW, Boot, CAL…) và so với BOM.

1. Chọn **Program**, rồi tick ECU cần đọc trong ô **ECU** (`All` hoặc từng ECU). Bảng được dựng lại theo service của các ECU đã chọn.
2. Đặt **Retry** (0 – 20): số lần đọc lại mỗi DID khi ECU không trả lời.
3. Bấm **Read**. Tool đọc tuần tự, mỗi lần một request; cột Status của từng dòng cho biết đang đọc, OK, Timeout hay NRC.
4. **Stop** hủy request đang chạy và bỏ các dòng còn lại.
5. **Export** xuất bảng ra Excel, giữ màu ô trạng thái.

Cột	Nội dung
Active	Tick để đọc dòng này
ECU Name, Type	ECU và loại thông tin (PN, HW, SW…)
Req ID, Resp ID	ID chẩn đoán của ECU
DID, Revision	DID chính và DID revision (nếu có)
Read	Giá trị đọc được: chữ ASCII theo luật của Program, nếu không có luật thì là hex. Dòng có revision hiện `<main>-<revision>`
Status	Kết quả đọc của dòng
Data from BOM	Giá trị BOM để so sánh, tô màu theo kết quả khớp / lệch

BOM Check

Nút **BOM Check** mở hộp thoại nạp dữ liệu BOM để điền cột Data from BOM. Hộp thoại mở sẵn Program đang chọn và file BOM mới nhất của Program đó.

- **Generate**: chọn sheet mẫu (Landscape / Portrait) của `Template_DID_BOM_Compare.xlsx`, tool tạo file BOM đã điền sẵn tên ECU của Program. Điền PN / Rev vào file này.
- **Load**: nạp file BOM có sẵn, dạng danh sách BOM (Common ECU Name, PN Type, DID No, VF PN kèm Rev) hoặc file mẫu đã điền. Tool quét toàn bộ file và hiện lên bảng.
- Bảng sửa được như Excel: copy / paste cả dòng hoặc cột, fill xuống / sang phải, thêm / xóa dòng, Undo / Redo. **Open Excel** mở file đang xem bằng Excel.
- Ô không áp dụng cho một ECU hiện xám và bị khóa. Ví dụ một ECU chỉ có **một** trong hai mục SW_TBOX (F164 / F149) hoặc FONT (F109 / F149), tùy service của ECU đó trong Program.
- **Save**: điền cột Data from BOM và so sánh ngay, kể cả khi đã Read trước đó. Dòng trạng thái báo kết quả, ví dụ `BOM CTX_BOM.xlsx: 4 entries, 4/4 row(s) matched`. Đóng hộp thoại mà không Save thì bảng giữ nguyên.
- Nếu chọn Program khác trong hộp thoại rồi Save, tool cảnh báo BOM không cùng Program với bảng.

7.4 DTC Reader (Diagnostics › DTC Reader)

DTC Reader đọc và xóa mã lỗi của nhiều ECU cùng lúc. Mỗi ECU chạy song song, nên tổng thời gian bằng ECU chậm nhất.

- Chọn **Program** và tick ECU (`All` hoặc từng ECU).
- **Mask**: status mask hex cho `19 01` / `19 02`, mặc định `09` (TestFailed + ConfirmedDTC), được nhớ cho lần sau.
- **Read** chạy cho mỗi ECU: `10 01` → `10 03` → `19 01 <mask>` → `19 02 <mask>`.
  - Không trả lời `10 01`: ghi **Offline**, dừng ECU đó. Trả NRC: ghi NRC, dừng ECU đó.
  - `10 03` lỗi vẫn đi tiếp.
  - `19 01` báo 0 lỗi: ghi **No DTC**, bỏ qua `19 02`.
  - Số DTC của `19 02` khác số báo ở `19 01` thì ghi `DTC Count Mismatch`.
- **Clear** hiện hộp xác nhận liệt kê ECU, rồi gửi `10 03` → `14 FF FF FF` và chờ `54`. ECU im lặng được thử lại tối đa 2 lần.
- **Stop** dừng tất cả ECU.
- **Export** xuất Excel gồm 3 sheet: Summary, DTC List, Raw.

Tab	Nội dung
Status	Mỗi bước lệnh một dòng: ECU, Command, Expect Data, Read Raw, Status Mask. Xanh = OK / No DTC, đỏ = có lỗi, cam = NRC, xám = Offline

Tab	Nội dung
Detail	Mỗi DTC một dòng: ECU, DTC Code, DTC Name, Repair Action, Status (ví dụ `0x09 → TestFailed + ConfirmedDTC`). ECU không có lỗi có một dòng No DTC / Offline / NRC

Tên lỗi và repair action lấy từ file `<ECU>_DTC.json` tạo ở Diagnostic Setting › DTC. ECU chưa có file này hiện "No DTC file for this ECU". Clear thành công thì Detail ghi `Cleared (54)`; thất bại thì giữ DTC đã đọc và ghi thêm lý do.

8. Công cụ (Tools)

Hai công cụ này làm việc offline, không cần phần cứng và không bị ảnh hưởng bởi Start / Stop.

8.1 DBC Editor (Tools › DBC Editor)

DBC Editor mở, sửa và lưu file DBC theo kiểu master-detail: chọn message thì bảng signal và hình bố trí bit đổi theo.

- **Messages**: danh sách message (ID, tên, DLC, node gửi, chu kỳ…), có ô lọc.
- **Signals**: signal của message đang chọn: start bit, length, byte order (Intel / Motorola), signed, factor, offset, min / max, đơn vị, node nhận. Start bit giữ đúng quy ước của DBC (Intel = bit thấp nhất, Motorola = bit cao nhất).
- **Signal Preview**: hình bố trí bit của message, thấy ngay signal chồng lấn.
- **Value Table**: double-click ô Value Table của signal để nhập bảng giá trị (ví dụ 0 = Off, 1 = On).
- **Attributes**: double-click ô Attributes để định nghĩa và gán attribute (ví dụ `GenMsgCycleTime`, `GenSigStartValue`), có kiểm tra giá trị hợp lệ.
- Khung bên phải có các tab **Network** (cây), **Nodes**, **DBC Preview** (xem nội dung file; nút Copy) và **Errors**. Nút `N errors` nhảy tới danh sách lỗi.

Khi lưu, các phần DBC mà editor không sửa (EV_, SIG_GROUP_, BA_REL_…) được giữ nguyên, cùng mã hóa ký tự và kiểu xuống dòng của file gốc. File lớn (hàng nghìn message, hàng chục nghìn signal) vẫn mở được.

8.2 CRC Calculator (Tools › CRC Calculator)

CRC Calculator tính CRC trên một chuỗi hex bất kỳ và giải thích từng bước tính.

- Dán chuỗi hex; CRC được tính lại ngay khi gõ. Chọn bộ tham số CRC (ví dụ CRC-8 SAE-J1850).

- **Chỉ tính đoạn**: chỉ tính trên byte [Từ byte … Đến byte], tính cả hai đầu, đếm từ 0 trên chuỗi đã dán.
- **So với byte**: so kết quả với byte tại một vị trí, hiện ✔ khớp hoặc ✘ lệch.
- **CDS ACU (0xF108)**: dán nguyên chuỗi phản hồi `62 F1 08 …`; tool tính trên byte 12 – 47 và so với byte 48.
- Tab **Các bước tính**: giá trị thanh ghi CRC trước, sau XOR và sau 8 lần dịch cho từng byte.
- Tab **Nguyên lý**: giải thích thuật toán theo tham số đang chọn, kèm ví dụ dịch từng bit với byte đầu tiên.

9. Kiểm thử (Test)

UDS Test chạy chuỗi lệnh chẩn đoán có so sánh kết quả PASS / FAIL; Program cho phép tự viết logic CAN bằng khối kéo thả hoặc Python.

9.1 UDS Test (Test › UDS Test)

Thanh trên cùng giống UDS Console: chọn **Program** và **ECU** (hoặc `(Manual)`), ô DLL Security Access, Tester Present ($3E) và sub-function. **Browse DLL** dùng một DLL khác cho phiên này mà không đổi Program. Rê chuột lên dòng trạng thái để xem toàn bộ cấu hình (bus, P2, Security, Tester Present).

Tab Sequence: soạn chuỗi test

1. Gõ tên vào ô Name rồi bấm **New** để tạo file sequence trong thư mục của Program; hoặc **Load** một sequence có sẵn.
2. Chọn service trong cây Services để thêm dòng.
3. Điền từng dòng theo bảng dưới. Ô sai tô đỏ, rê chuột để xem lý do.
4. **Save** / **Save As**. Đổi sequence khi chưa lưu thì tool hỏi trước.

Cột	Ý nghĩa
Service	SID của bước. Ô tick = bước được bật; RUN bỏ qua bước không tick
Function	Các byte sau SID, ví dụ `22` + `F1 90` = request `22 F1 90`
Expected Response	Phản hồi mong đợi; tự điền theo Service + Function cho tới khi bạn gõ giá trị riêng (chữ nghiêng = tự điền)
Times	Thời gian chờ hoặc gửi lặp (cú pháp ở Phụ lục)

Cột	Ý nghĩa
Comment	Ghi chú

Phím tắt: chuột phải hoặc Ctrl+D nhân đôi dòng, Alt+Lên / Alt+Xuống di chuyển, Del xóa, Ctrl+E bật / tắt.

Tab DialogBox: chạy test

- **Sequence List** liệt kê sequence của Program. Click để hiện lên bảng, double-click để mở sửa.
- Nút ở cột **Send** gửi riêng từng bước; dòng cyclic có nút Start / Stop.
- **RUN** chạy cả sequence; chuột phải chọn **Run from this step** để chạy từ một bước.
- **STOP** (hoặc phím Esc) dừng ngay request đang chờ, mọi dòng cyclic và các lệnh Send đang xếp hàng.
- Cột **Result** ghi ví dụ `PASS · 23 ms` hoặc `FAIL — NRC 0x31 requestOutOfRange`, tô màu theo kết quả. Thanh trạng thái tổng kết, ví dụ `Done — PASS 5 · FAIL 1 · skipped 1 · cyclic 2`.
- **Save results** xuất Excel (sheet Results và Log), CSV hoặc file log.

Cần biết khi chạy:

- Các nút chỉ bật sau Home › Start và khi có Diagnostic Bus.
- Mỗi bước chỉ gửi một lần (không tự gửi lại), nên timeout được ghi thành kết quả.
- Dòng cyclic vẫn chạy sau khi RUN xong, cho tới khi bấm STOP, để giữ session cho các lệnh gửi tay sau đó.
- Dòng `3E` cyclic dùng Tester Present của engine: tự tạm dừng khi có request khác, không làm chậm sequence.
- Bước `27 01` với ECU có DLL: tool tự tính và gửi key; bước chỉ PASS khi key được chấp nhận (cột Actual hiện thêm `| key → 67 02`). Seed toàn `00` được hiểu là ECU đã mở khóa.
- Không chạy UDS Test cùng lúc với UDS Console trên cùng ECU.

9.2 Program (Test › Program)

Program là nơi viết logic chạy trên Measure Bus: mô phỏng ECU, gateway, gửi / nhận message, gọi lệnh UDS. Có thể viết bằng khối kéo thả (tab Block) hoặc Python (tab Code), và xuất sang CAPL để dùng trong CANoe.

Quản lý file: **New / Open / Save** một chương trình (lưu thành `<tên>.py` và `<tên>.layout.json`). Tool hỏi trước khi bỏ thay đổi chưa lưu.

Tab Block

- Palette có các nhóm khối: Sự kiện, CAN, Timer, Điều khiển, Biến, Toán tử, Hàm, Log, UDS, Test Sequence, Python.
- Kéo khối vào canvas hoặc double-click để thêm dưới khối đang chọn. Thả gần chỗ nối thì khối tự hít vào, có vạch báo trước. Ô điều kiện chỉ nhận khối kiểu đúng / sai.
- Bấm vào trường để gõ giá trị; trường có ▾ mở danh sách message / signal của DBC, timer, biến, hàm, ECU của Diagnostic Setting.
- Delete xóa khối; kéo một chồng khối thả lên Palette cũng xóa. Ctrl+Z / Ctrl+Y để Undo / Redo.
- Kéo nền để chọn nhiều khối; chuột phải hoặc chuột giữa để kéo canvas; Ctrl + cuộn để zoom; **Fit** để thấy toàn bộ.
- Panel Property bên phải sửa được các trường của khối đang chọn và hiện code Python tương ứng.

Tab Code

- Trình soạn thảo có số dòng, tô màu cú pháp, Ctrl+/ để comment, Ctrl + cuộn để zoom. Có thể chia code thành nhiều tab.
- Tool phân tích code ngay khi gõ: danh sách handler, biến và timer, tab Issue báo lỗi kèm dòng.
- Double-click một mục trong Palette để chèn đoạn code mẫu.
- Ví dụ handler: `@on_message("*")` nhận mọi message; `forward()` chuyển tiếp message (gateway); `write()` ghi ra tab Log; `e2e_message()` khai báo vị trí CRC / Alive Counter cho một message. Message có thể ghi kèm network, ví dụ `"CAN1::Msg"`.

Đồng bộ Block ↔ Code: chồng khối nào sửa ở tab Block được ghi sang tab Code khi mở tab Code, Run, Validate hoặc Save; phần code còn lại giữ nguyên, kể cả comment. Một handler bị sửa ở cả hai nơi thì tool hỏi chọn bản nào. Code viết tay cũng nhập ngược thành khối được.

Chạy

- **Run** cần Home › Start và chương trình không có lỗi. Trong lúc chạy, trình soạn thảo chỉ đọc.
- Tab Log hiện nội dung `write()`; lỗi lúc chạy vào tab Issue. **Var & Time › Current** hiện giá trị biến và timer, cập nhật mỗi 200 ms.
- Bật **Debug** trước khi Run để thấy khối / dòng code vừa chạy sáng lên.
- Panel cho phép sửa biến và signal ngay khi đang chạy; tab Bus hiện trạng thái bus trực tiếp.
- **Stop** (hoặc Home › Stop, hoặc đóng cửa sổ) dừng chương trình; các handler `on_stop` chạy trước.

Xuất CAPL: chọn target CAPL để thấy trước phần nào không xuất được (khối chỉ chạy Python có dấu ⚠), rồi **Export CAPL**.

10. Trợ giúp (Help)

Help › About cho biết phiên bản và thông tin hệ thống cần gửi kèm khi báo lỗi; **Help › License…** quản lý key (xem mục 2).

Tab của About	Nội dung
About	Phiên bản và build, nhà phát triển, bản quyền, điều khoản sử dụng, giấy phép mã nguồn mở, liên hệ hỗ trợ (email, link)
Features	Các chức năng của tool, nhóm theo menu
System	Windows, Python, Qt, các thư viện; trạng thái `vxlapi64.dll` và `PCANBasic.dll` trong thư mục `Lib` (thiếu thì hiện chữ đỏ); Machine ID
Third-Party	Thư viện bên thứ ba và giấy phép của chúng

Nút **Copy System Info** chép toàn bộ thông tin dạng chữ để dán vào email hoặc báo cáo lỗi.

11. Xử lý sự cố thường gặp

Phần lớn sự cố đến từ mapping kênh hoặc chưa Start; xem System Messages trước, vì tool ghi lý do ở đó.

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Start không mở network nào	Chưa có network nào Active = Yes trong Channel Mapping	Mở Channel Mapping, chọn thiết bị và đặt Active = Yes
Một network bị bỏ qua khi Start	Thiết bị đã map chưa cắm hoặc đổi cổng	Cắm lại, bấm Scan Devices trong Channel Mapping
Thiết bị hiện xám trong bảng Diagnostic	Thiết bị không phải Vector đã dùng ở Measure Bus	Dùng thiết bị khác cho chẩn đoán, hoặc dùng kênh Vector
Không sửa được Channel Mapping	Đang chạy	Home › Stop rồi sửa
Đổi baudrate / P2 không có tác dụng	Thay đổi chỉ áp dụng ở lần Start sau	Home › Stop rồi Start lại

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
Home › Start bị khóa	Bus Monitor đang Playback	Dừng Playback hoặc đóng Bus Monitor
UDS Console: nút Send xám	Chưa Start hoặc chưa map Diagnostic Bus	Map Diagnostic Bus, rồi Home › Start
Mọi request đều Timeout	Sai Req / Resp ID, sai network, sai CAN type, hoặc ECU chưa bật nguồn	Gõ `?` trong UDS Console để xem cấu hình đang dùng và kiểm tra lại
Kết quả chẩn đoán lẫn lộn giữa hai cửa sổ	Hai cửa sổ chẩn đoán cùng chạy trên một ECU	Chỉ để một cửa sổ chạy trên ECU đó
Không tự gửi key Security Access	Không tìm thấy DLL hoặc bộ tính key chưa sẵn sàng	Đọc lý do trong Console; sửa đường dẫn DLL trong Diagnostic Setting (ô đỏ)
DTC Reader: "No DTC file for this ECU"	Chưa tạo file DTC cho ECU	Diagnostic Setting › DTC › chọn file ODX › Generate
DTC Reader: "DTC Count Mismatch"	Số lỗi của `19 01` và `19 02` khác nhau	Lưu hex thô ở tab Status và gửi cho bộ phận hỗ trợ
Dòng Trace chuyển xám	ECU đã ngừng gửi ID đó	Kiểm tra ECU / nguồn; dòng sáng lại khi có frame mới
Graphic: signal xám "not found"	DBC hiện tại không còn signal trong bộ đã lưu	Nạp đúng DBC cho network
Mục menu mờ có 🔒	License không có chức năng này	Help › License… để xem gói, liên hệ nhà cung cấp
Tool mở chậm	Một bước khởi động mất nhiều thời gian	Gửi file `startup.log` (mục 2)

Khi báo lỗi, gửi kèm nội dung **Help › About › Copy System Info**, ảnh chụp màn hình và file `startup.log`.

12. Phụ lục

A. Thư mục dữ liệu

Dữ liệu chẩn đoán và thư viện Sequence Check nằm trong thư mục cài MS_Tool, dùng chung cho mọi project. Tên thư mục Program chính là tên Program hiện trong các cửa sổ chẩn đoán.

```
<thư mục MS_Tool>/
├─ Lib/                                vxlapi64.dll, PCANBasic.dll
├─ Projects/                           các project (File › Open)
├─ Diagnostics Setting/
│  └─ <Program>/
│     ├─ <Program>_DID_Config.json     ECU, Req / Resp ID, service, DLL
│     ├─ <Program>_ASCIIRule.json      luật dịch ASCII
│     └─ <Program>_DTC/
│        └─ <ECU>_DTC.json             mã lỗi, tên lỗi, repair action
├─ Sequence Check/<tên>.json           sequence của Graphic › Configurations
├─ Template_DID_BOM_Compare.xlsx       mẫu BOM cho DID Reader › BOM Check
└─ license.lic                         (tùy chọn) tự kích hoạt lần đầu

%APPDATA%\MS_Tool\logs\startup.log     thời gian các bước khởi động
```

B. Định dạng file

Đuôi file	Dùng ở	Ghi chú
`.dbc`	CAN Database, DBC Editor	Cơ sở dữ liệu message / signal
`.blf`, `.asc`, `.txt`	Bus Monitor › Log, Playback	Ghi và phát lại; `.txt` là định dạng của MS_Tool
`.trc`, `.csv`, `.log`, `.mf4`, `.db`	Bus Monitor › Playback	Log của PEAK, python-can, candump, ASAM MDF, SQLite
`.meta.json`	Đi kèm file log	Tên network của từng kênh
`.json`	Send CAN (Export), Sequence Check, UDS Test, Program	Hàng đợi TX, sequence, chuỗi test, bố cục khối
`.py`	Test › Program	Code của chương trình
`.xlsx`	DID / DTC Reader (Export), UDS Test (Save results), BOM, ODX	Báo cáo và dữ liệu đầu vào
`.lic`	License	File key

C. Cú pháp UDS Test

Expected Response	Nghĩa
`62 F1 90 *`	Các byte cụ thể; `XX` hoặc `?` = một byte bất kỳ; `*` ở cuối = các byte sau tùy ý

Expected Response	Nghĩa
(để trống)	Phản hồi dương bất kỳ
`*`	Nhận mọi kết quả, chỉ ghi lại
`NO RESP`	Không có phản hồi (sub-function có bit 0x80, hoặc ECU im lặng)
`NRC`	Phản hồi âm bất kỳ
`NRC 31` hoặc `7F 22 31`	Đúng NRC đó

Times	Nghĩa
(để trống)	Chạy bước tiếp ngay sau khi có phản hồi
`delay(100)`	Chờ 100 ms sau bước này rồi mới chạy bước tiếp
`cyclic(2000)`	Từ đây gửi bước này mỗi 2000 ms ở nền và đi tiếp các bước sau, cho tới khi STOP

D. Service UDS thường dùng

Phản hồi dương = SID + 0x40 (ví dụ `22` → `62`); phản hồi âm = `7F <SID> <NRC>`.

SID	Service	Ví dụ
`10`	DiagnosticSessionControl	`10 01` default, `10 03` extended
`11`	ECUReset	`11 01` hard reset
`14`	ClearDiagnosticInformation	`14 FF FF FF` xóa mọi DTC
`19`	ReadDTCInformation	`19 01 09` đếm DTC, `19 02 09` đọc DTC
`22`	ReadDataByIdentifier	`22 F1 90` đọc VIN
`27`	SecurityAccess	`27 01` xin seed, `27 02` gửi key
`2E`	WriteDataByIdentifier	`2E F1 90 …` ghi VIN
`31`	RoutineControl	`31 01 <RID>` chạy routine
`3E`	TesterPresent	`3E 00`, `3E 80` (không cần phản hồi)

E. Mã NRC thường gặp (ISO 14229-1)

NRC	Ý nghĩa
`10`	generalReject
`11`	serviceNotSupported
`12`	subFunctionNotSupported
`13`	incorrectMessageLengthOrInvalidFormat
`22`	conditionsNotCorrect
`24`	requestSequenceError
`31`	requestOutOfRange
`33`	securityAccessDenied
`35`	invalidKey
`36`	exceededNumberOfAttempts
`37`	requiredTimeDelayNotExpired
`78`	responsePending (ECU đang xử lý; tool chờ thêm theo P2*)
`7E`	subFunctionNotSupportedInActiveSession
`7F`	serviceNotSupportedInActiveSession

F. Bit trạng thái DTC

Mask `09` = bit 0 + bit 3 (testFailed + confirmedDTC).

Bit	Tên
0	testFailed
1	testFailedThisOperationCycle
2	pendingDTC
3	confirmedDTC
4	testNotCompletedSinceLastClear
5	testFailedSinceLastClear

Bit	Tên
6	testNotCompletedThisOperationCycle
7	warningIndicatorRequested